

GIẢI QUYẾT VẤN ĐỀ VÀ TÌM KIẾM (PROBLEM SOLVING AND SEARCH)

CHƯƠNG 3

Nhắc nhở

Bài tập 0 hạn nộp lúc 5 giờ chiều hôm nay

Bài tập 1 đã đang, hạn nộp ngày 9/2

Phần 105 sẽ chuyển sang 9-10h sáng bắt đầu từ tuần sau

Phác thảo

- ◇ Tác nhân giải quyết vấn đề
- ◇ Các loại sự cố
- ◇ Xây dựng bài toán
- ◇ Các vấn đề mẫu
- ◇ Thuật toán tìm kiếm cơ bản

Tác nhân giải quyết vấn đề

Hình thức tổng đại lý bị hạn chế:

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
static: seq, an action sequence, initially empty
         state, some description of the current world state
         goal, a goal, initially null
         problem, a problem formulation

state ← UPDATE-STATE(state, percept)
if seq is empty then
  goal ← FORMULATE-GOAL(state)
  problem ← FORMULATE-PROBLEM(state, goal)
  seq ← SEARCH(problem)
  action ← RECOMMENDATION(seq, state)
  seq ← REMAINDER(seq, state)
return action
```

Lưu ý: đây là cách giải quyết vấn đề ngoại tuyến; giải pháp được thực thi “

nhắm mắt lại.”

Trực tuyến giải quyết vấn đề liên quan đến hành động mà không có kiến thức đầy đủ.

Ví dụ: Romania

Đang đi nghỉ ở Romania; hiện đang ở Arad.

Chuyến bay khởi hành vào ngày mai từ Bucharest

Xây dựng mục tiêu:

ở Bucharest

Xây dựng bài toán:

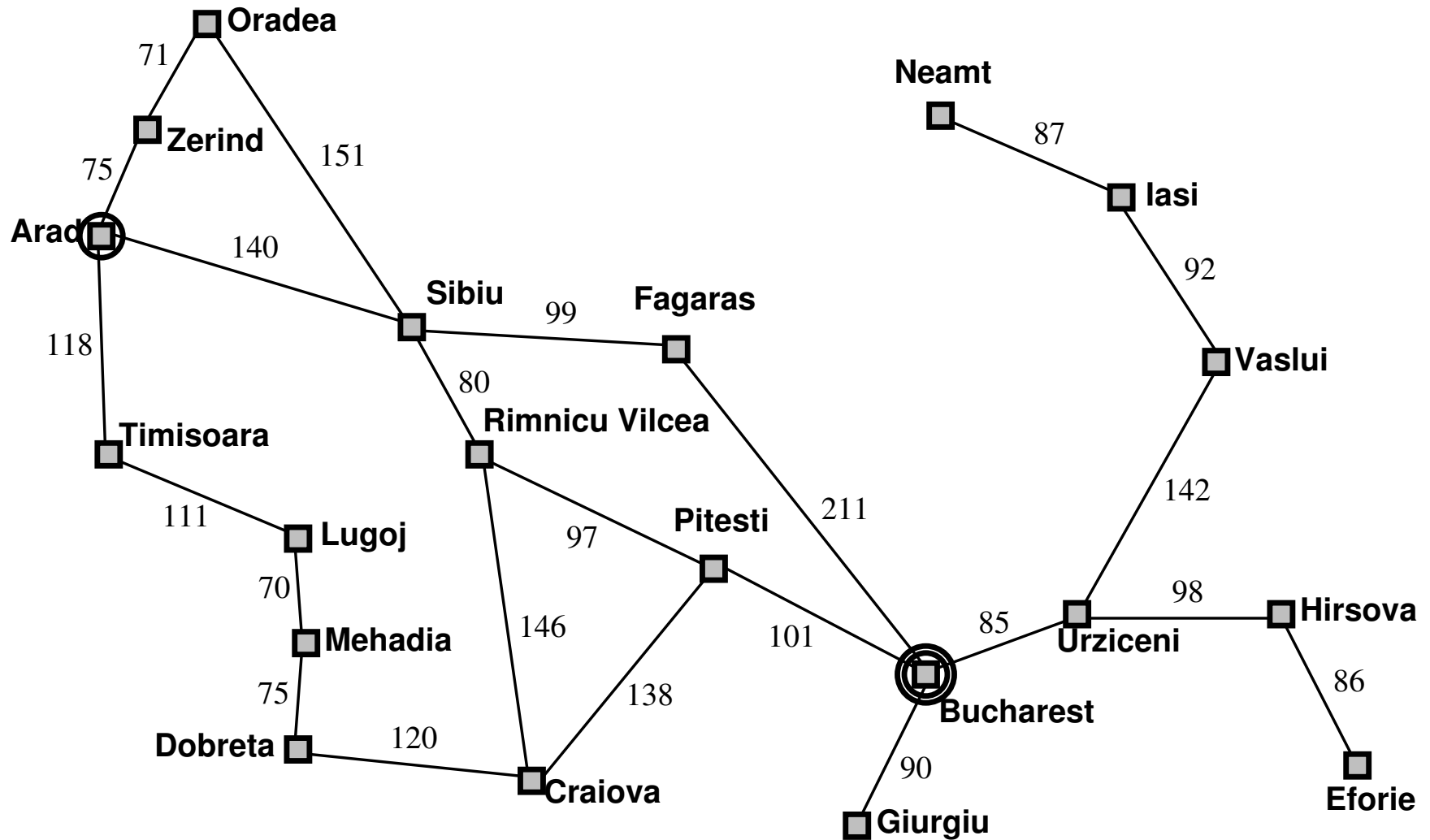
tiểu bang: các thành phố khác nhau

hành động: lái xe giữa các thành phố

Tìm giải pháp:

chuỗi các thành phố, ví dụ: Arad, Sibiu, Fagaras, Bucharest

Ví dụ: Romania



Các loại sự cố

Có tính xác định, hoàn toàn có thể quan sát được $\implies$ vấn đề một trạng thái

Đại lý biết chính xác nó sẽ ở trạng thái nào; giải pháp là một chuỗi

Không thể quan sát được $\implies$ vấn đề về tuân thủ

Đại lý có thể không biết nó ở đâu; nghiệm (nếu có) là dãy

Không xác định và/hoặc có thể quan sát được một phần $\implies$ vấn đề dự phòng

nhận thức cung cấp thông tin **mới** về trạng thái hiện tại

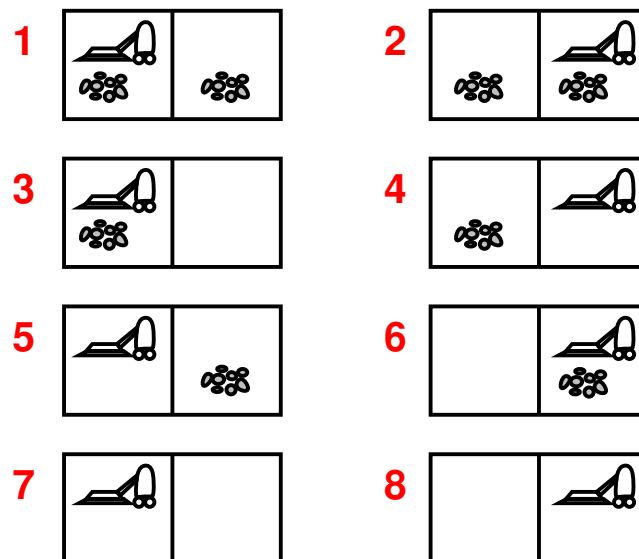
giải pháp là kế hoạch dự phòng hoặc chính sách

thường **xen kẽ** tìm kiếm, thực thi

Không gian trạng thái không xác định $\implies$ vấn đề khám phá (“trực tuyến”)

Ví dụ: thế giới chân không

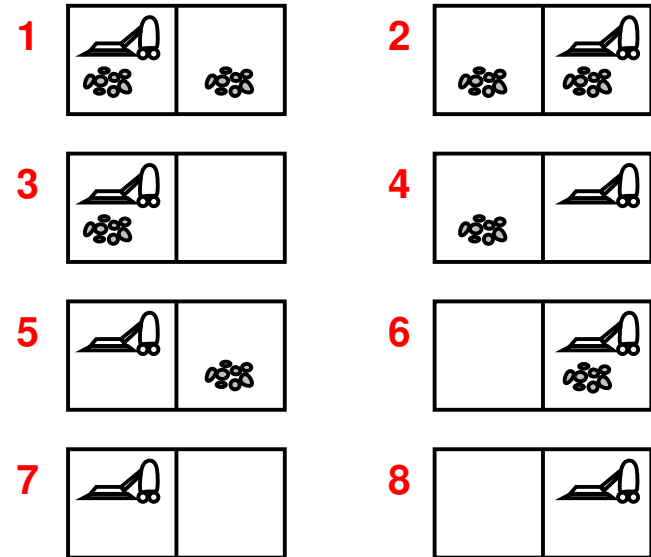
Trạng thái đơn, bắt đầu bằng #5. Giải pháp??



Ví dụ: thể giới chân không

Trạng thái đơn, bắt đầu bằng #5. Giải pháp??
 [Right, Suck]

Conformant, bắt đầu trong
 {1, 2, 3, 4, 5, 6, 7, 8}
 ví dụ: *Right* chuyển đến {2, 4, 6, 8}.
Giải pháp??



Ví dụ: thế giới chân không

Trạng thái đơn, bắt đầu bằng #5. Giải pháp??
 [*Right, Suck*]

Conformant, bắt đầu trong
 {1, 2, 3, 4, 5, 6, 7, 8}

ví dụ: *Right* chuyển đến {2, 4, 6, 8}.

Giải pháp??

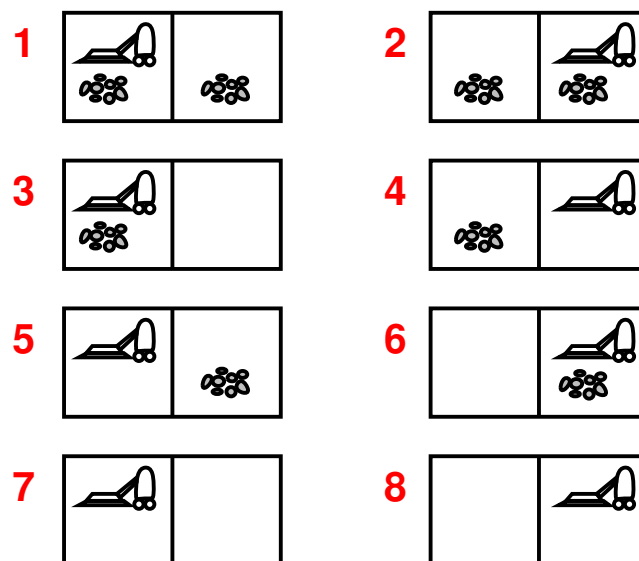
[*Right, Suck, Left, Suck*]

Dự phòng, bắt đầu trong #5

Định luật Murphy: *Suck* có thể làm bẩn một
 tấm thảm sạch

Cảm biến cục bộ: bụi bẩn, chỉ vị trí.

Giải pháp??



Ví dụ: thể giới chân không

Trạng thái đơn, bắt đầu bằng #5. Giải pháp??
 [Right, Suck]

Conformant, bắt đầu trong
 {1, 2, 3, 4, 5, 6, 7, 8}

ví dụ: *Right* chuyển đến {2, 4, 6, 8}.

Giải pháp??

[Right, Suck, Left, Suck]

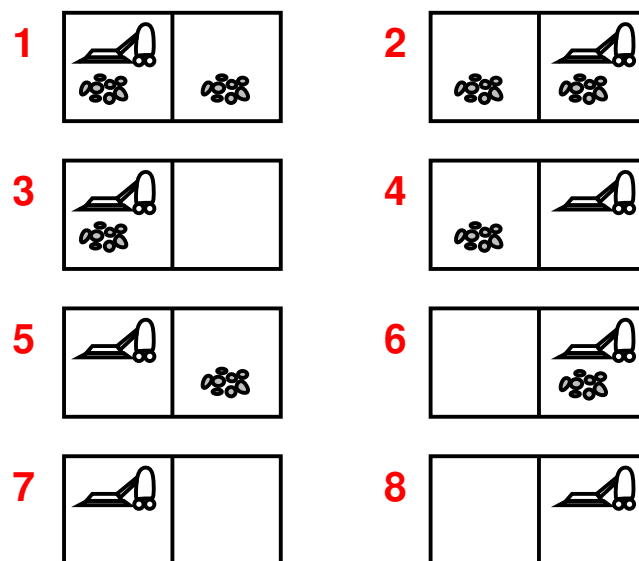
Dự phòng, bắt đầu trong #5

Định luật Murphy: *Suck* có thể làm bẩn một
 tấm thảm sạch

Cảm biến cục bộ: bụi bẩn, chỉ vị trí.

Giải pháp??

[Right, **if dirt then Suck**]



Xây dựng bài toán trạng thái đơn

Sự cố được xác định bởi bốn mục:

trạng thái ban đầu ví dụ: “tại Arad”

hàm kế tiếp $S(x)$ = tập hợp các cặp hành động–trạng thái

ví dụ: $S(Arad) = \{\langle Arad \rightarrow Zerind, Zerind \rangle, \dots\}$

kiểm tra mục tiêu, có thể là

rõ ràng, ví dụ: $x =$ “tại Bucharest”

ẩn, ví dụ: $NoDirt(x)$

chi phí đường dẫn (phụ gia)

ví dụ: tổng khoảng cách, số hành động được thực hiện, v.v.

$c(x, a, y)$ là chi phí bước, giả định là ≥ 0

Giải pháp là một chuỗi hành động
dẫn từ trạng thái ban đầu đến trạng thái mục tiêu

Chọn không gian trạng thái

Thế giới thực phức tạp đến mức ngớ ngẩn

Không gian trạng thái $\Rightarrow$ phải được **trừu tượng** để giải quyết vấn đề

(Tóm tắt) trạng thái = tập hợp các trạng thái thực

(Tóm tắt) hành động = sự kết hợp phức tạp của các hành động thực tế

ví dụ: “Arad $\rightarrow$ Zerind” đại diện cho một tập hợp phức tạp

về các tuyến đường có thể, đường vòng, điểm dừng nghỉ, v.v.

Để đảm bảo khả năng thực hiện được, **bất kỳ** trạng thái thực “ở Arad”

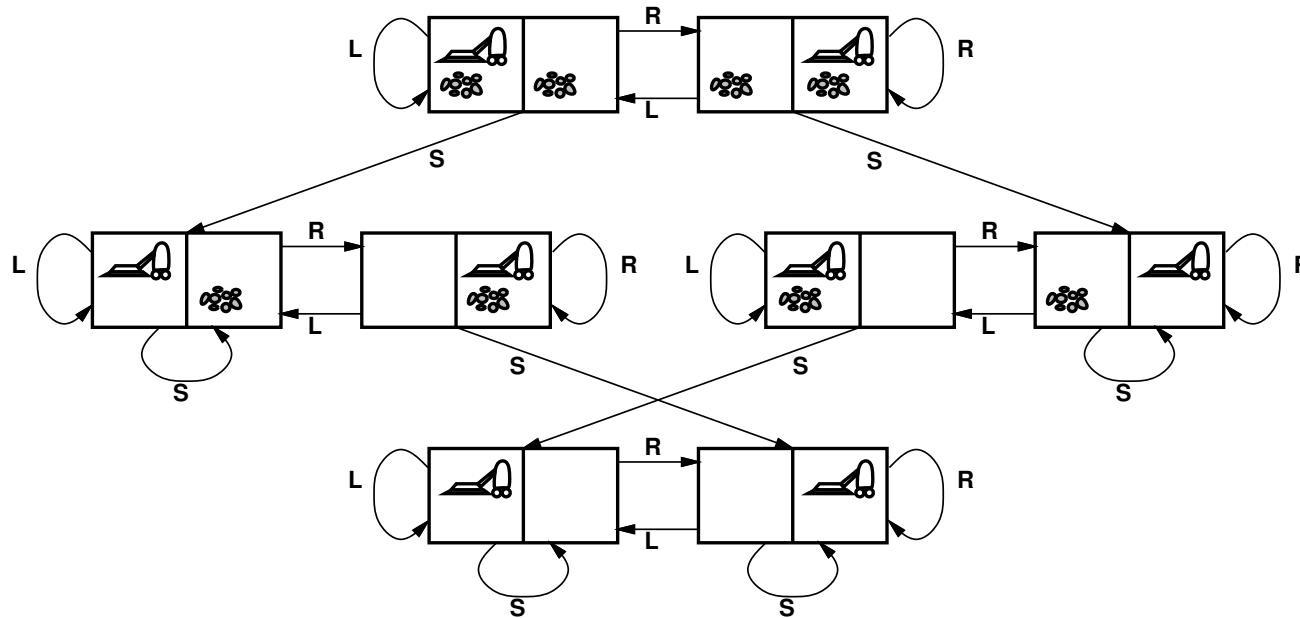
phải đến **some** trạng thái thực “ ở Zerind ”

(Tóm tắt) giải pháp =

tập hợp các đường dẫn thực sự là giải pháp trong thế giới thực

Mỗi hành động trừu tượng phải “dễ dàng hơn” so với vấn đề ban đầu!

Ví dụ: đồ thị không gian trạng thái thế giới chân không



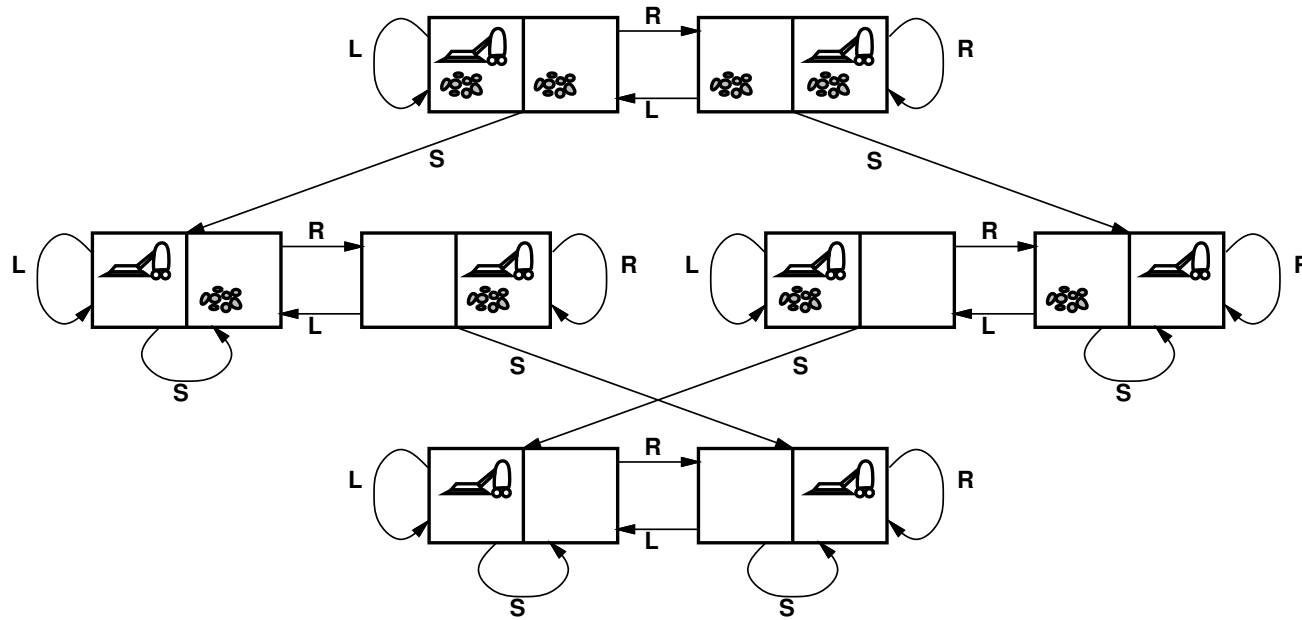
trạng thái??

hành động??

kiểm tra mục tiêu??

chi phí đường đi??

Ví dụ: đồ thị không gian trạng thái thế giới chân không



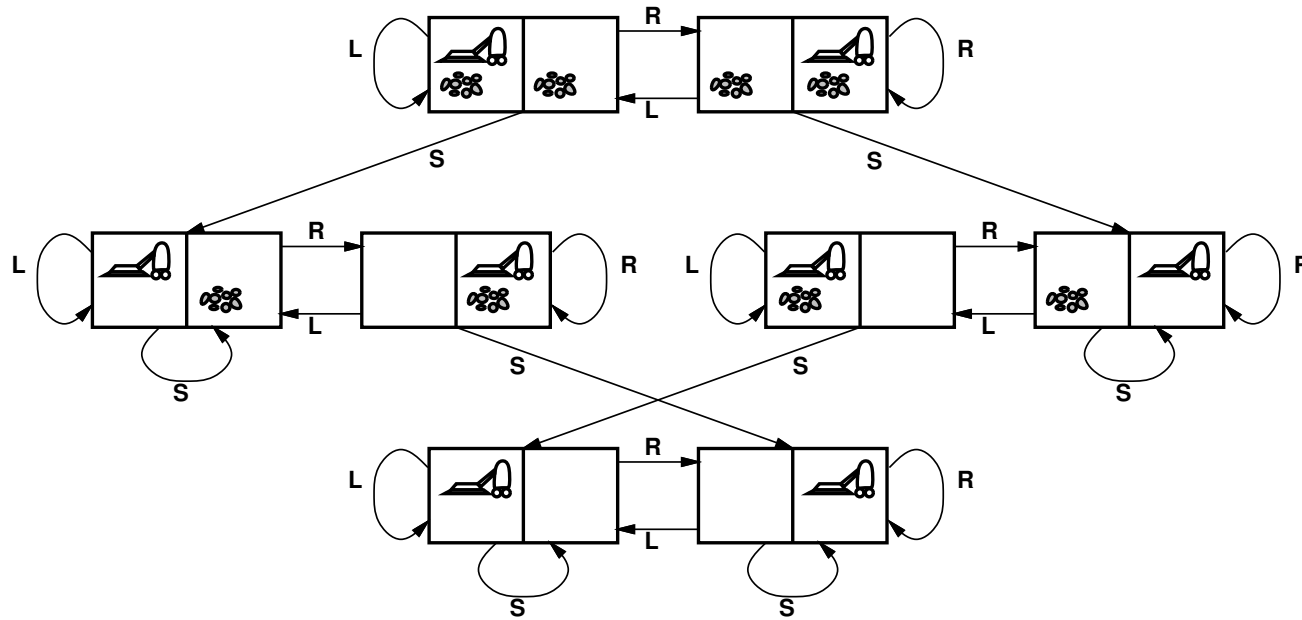
state??: số nguyên bụi bẩn và vị trí robot (bỏ qua bụi bẩn **số lượng**, v.v.)

hành động??

kiểm tra mục tiêu??

chi phí đường dẫn??

Ví dụ: đồ thị không gian trạng thái thế giới chân không



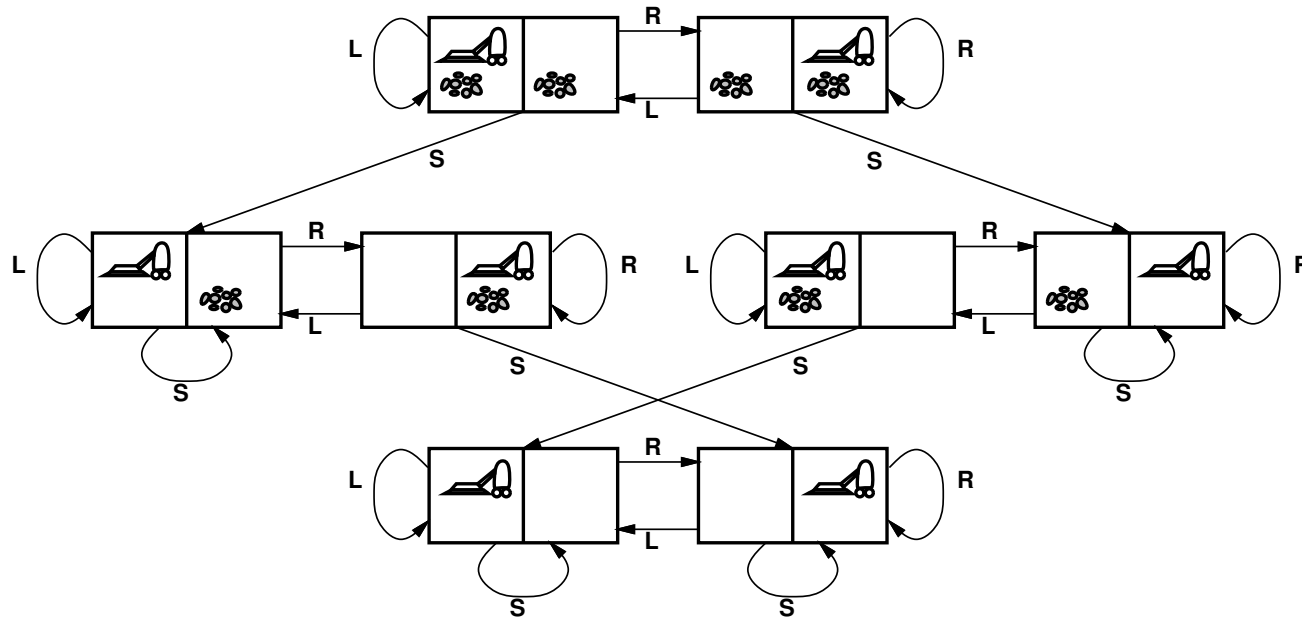
state??: số nguyên bụi bẩn và vị trí robot (bỏ qua bụi bẩn số lượng, v.v.)

hành động??: *Left*, *Right*, *Suck*, *NoOp*

kiểm tra mục tiêu??

chi phí đường dẫn??

Ví dụ: đồ thị không gian trạng thái thế giới chân không



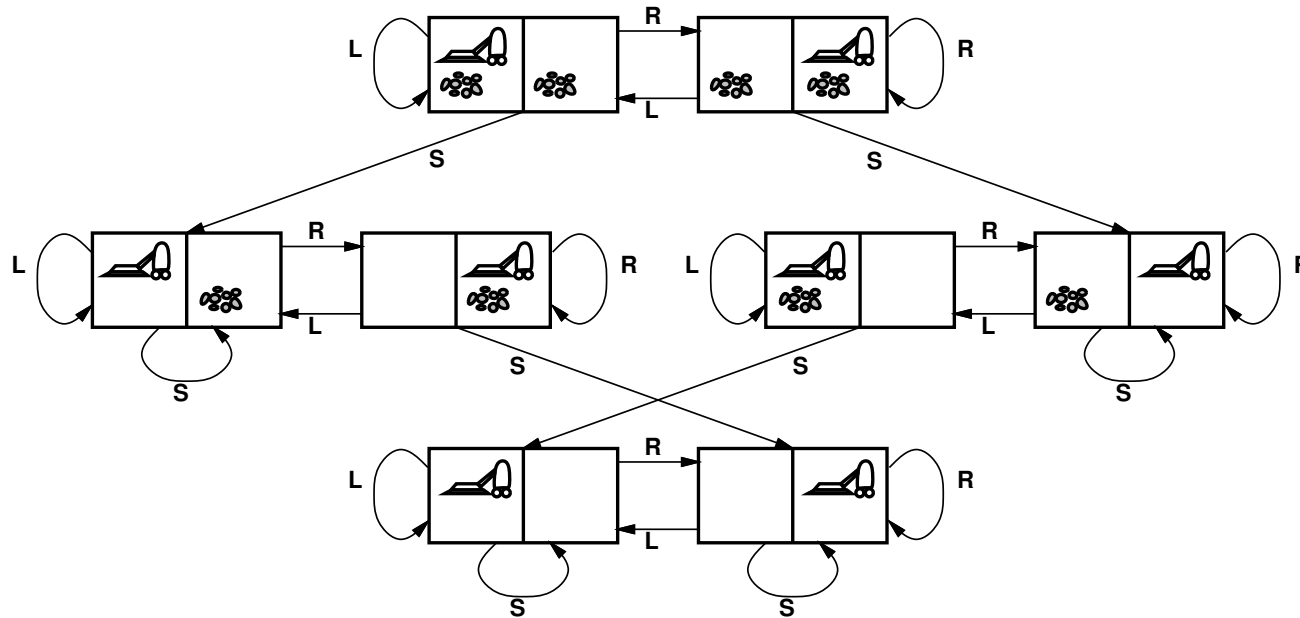
state??: số nguyên bụi bẩn và vị trí robot (bỏ qua bụi bẩn số lượng, v.v.)

hành động??: *Left*, *Right*, *Suck*, *NoOp*

kiểm tra mục tiêu??: không có bụi bẩn

chi phí đường dẫn??

Ví dụ: đồ thị không gian trạng thái thế giới chân không



state??: số nguyên bụi bẩn và vị trí robot (bỏ qua bụi bẩn **số lượng**, v.v.)

hành động??: *Left*, *Right*, *Suck*, *NoOp*

kiểm tra mục tiêu??: không có bụi bẩn

chi phí đường dẫn??: 1 cho mỗi hành động (0 cho *NoOp*)

Ví dụ: Câu đố 8 ô

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

trạng thái??

hành động??

kiểm tra mục tiêu??

chi phí đường đi??

Ví dụ: Câu đố 8 ô

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

state??: vị trí số nguyên của các ô (bỏ qua các vị trí trung gian)

hành động??

kiểm tra mục tiêu??

chi phí đường đi??

Ví dụ: Câu đố 8 ô

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

state??: vị trí số nguyên của các ô (bỏ qua các vị trí trung gian)

actions??: di chuyển trống sang trái, phải, lên, xuống (bỏ qua việc gõ nhiều, v.v.)

kiểm tra mục tiêu??

chi phí đường đi??

Ví dụ: Câu đố 8 ô

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

state??: vị trí số nguyên của các ô (bỏ qua các vị trí trung gian)

actions??: di chuyển trống sang trái, phải, lên, xuống (bỏ qua việc gõ nhầm, v.v.)

kiểm tra mục tiêu??: = trạng thái mục tiêu (đã cho)

chi phí đường đi??

Ví dụ: Câu đố 8 ô

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

state??: vị trí số nguyên của các ô (bỏ qua các vị trí trung gian)

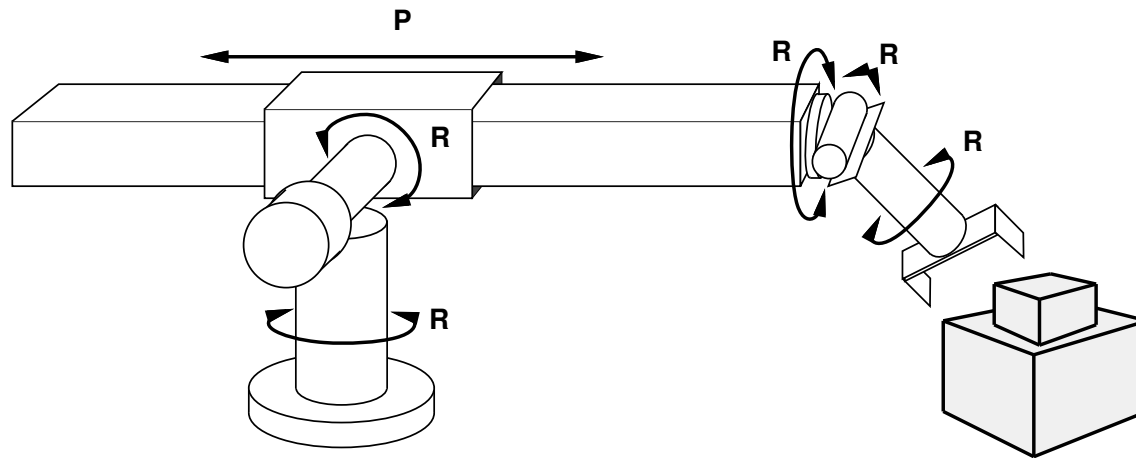
actions??: di chuyển trống sang trái, phải, lên, xuống (bỏ qua việc gõ nhầm, v.v.)

kiểm tra mục tiêu??: = trạng thái mục tiêu (đã cho)

chi phí đường dẫn??: 1 mỗi lần di chuyển

[Lưu ý: giải pháp tối ưu của n -Puzzle là NP-hard]

Ví dụ: lắp ráp robot



state??: tọa độ có giá trị thực của các góc khớp robot

các bộ phận của đối tượng được lắp ráp

hành động??: chuyển động liên tục của các khớp robot

kiểm tra mục tiêu??: lắp ráp hoàn chỉnh **không bao gồm robot!**

chi phí đường dẫn??: thời gian thực hiện

Thuật toán tìm kiếm cây

Ý tưởng cơ bản:

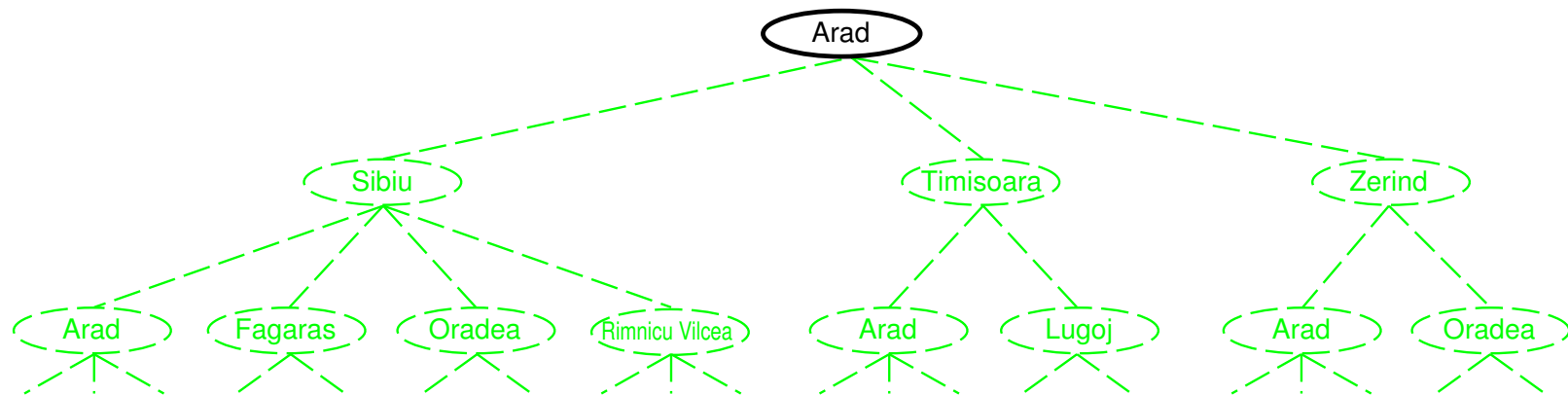
ngoại tuyến, mô phỏng khám phá không gian trạng thái

bằng cách tạo ra những trạng thái kế thừa đã được khám phá

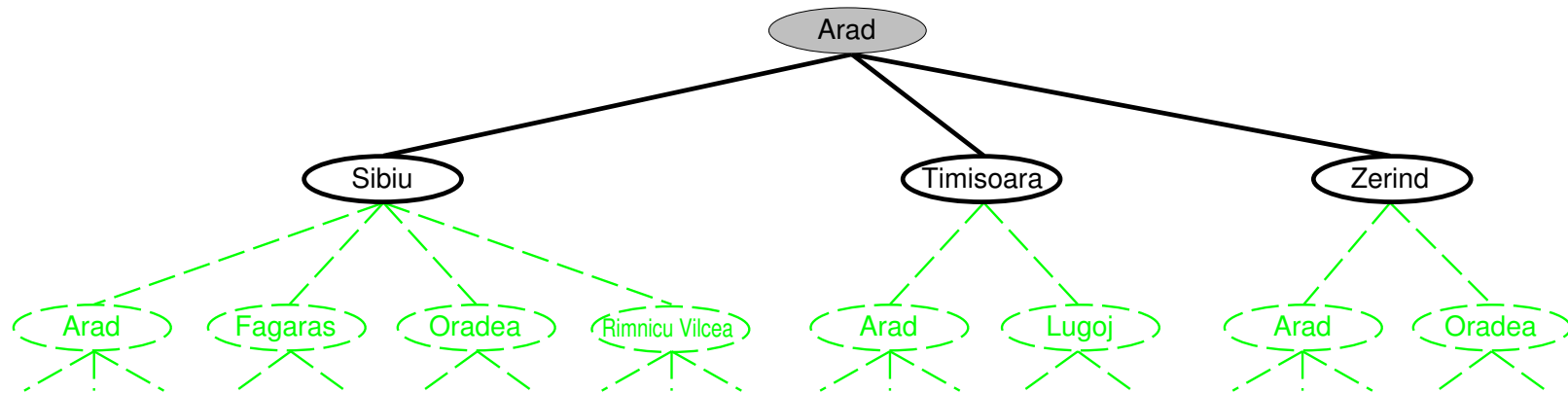
(còn gọi là trạng thái **mở rộng**)

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
loop do
  if there are no candidates for expansion then return failure
  choose a leaf node for expansion according to strategy
  if the node contains a goal state then return the corresponding solution
  else expand the node and add the resulting nodes to the search tree
end
```

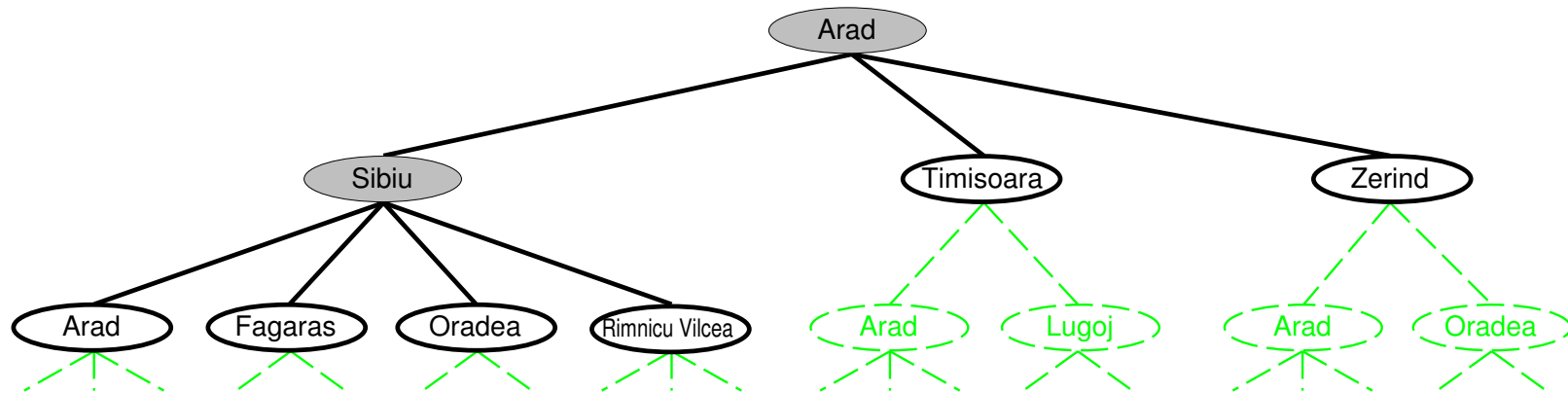
Ví dụ tìm kiếm cây



Ví dụ tìm kiếm cây



Ví dụ tìm kiếm cây



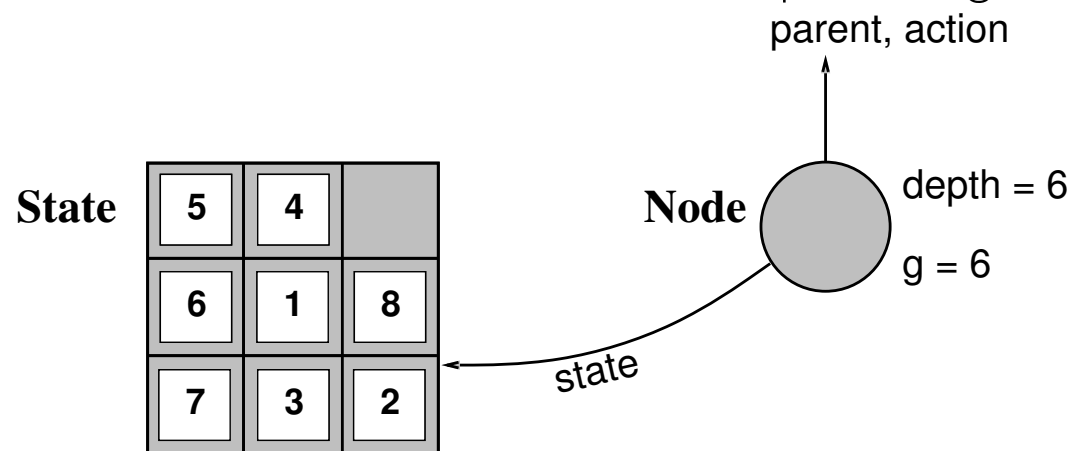
Triển khai: trạng thái so với. nút

Trạng thái là (biểu thị của) cấu hình vật lý

Nút là cấu trúc dữ liệu cấu thành một phần của cây tìm kiếm

bao gồm parent, children, deep, path cost $g(x)$

Hoa không có cha mẹ, con cái, độ sâu, hoặc chi phí đường đi!



Hàm **EXPAND** tạo các nút mới, điền vào các nút khác nhau các trường và sử dụng **SUCCESSORFN** của bài toán để tạo ra các trạng thái tương ứng.

Thực hiện: tìm kiếm cây tổng quát

```
function TREE-SEARCH(problem, fringe) returns a solution, or failure
fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
loop do
  if fringe is empty then return failure
  node ← REMOVE-FRONT(fringe)
  if GOAL-TEST(problem, STATE(node)) then return node
  fringe ← INSERTALL(EXPAND(node, problem), fringe)
```

```
function EXPAND(node, problem) returns a set of nodes
successors ← the empty set
for each action, result in SUCCESSOR-FN(problem, STATE[node]) do
  s ← a new NODE
  PARENT-NODE[s] ← node; ACTION[s] ← action; STATE[s] ← result
  PATH-COST[s] ← PATH-COST[node] + STEP-COST(STATE[node], action, result)
  DEPTH[s] ← DEPTH[node] + 1
  add s to successors
return successors
```

Chiến lược tìm kiếm

Chiến lược được xác định bằng cách chọn thứ tự **mở rộng nút**

Các chiến lược được đánh giá theo các khía cạnh sau:

tính đầy đủ—nó có luôn tìm ra giải pháp nếu có không?

độ phức tạp về thời gian—số lượng nút được tạo/mở rộng

Độ phức tạp của không gian—số lượng nút tối đa trong bộ nhớ

optimality—nó có luôn tìm ra giải pháp có chi phí thấp nhất không?

Độ phức tạp về thời gian và không gian được đo bằng

b —hệ số phân nhánh tối đa của cây tìm kiếm

d —độ sâu của giải pháp chi phí thấp nhất

m —độ sâu tối đa của không gian trạng thái (có thể là ∞)

Chiến lược tìm kiếm không chính xác

Chiến lược **Uninformed** chỉ sử dụng thông tin có sẵn trong định nghĩa vấn đề

Tìm kiếm theo chiều rộng

Tìm kiếm chi phí thống nhất

Tìm kiếm theo chiều sâu

Tìm kiếm giới hạn độ sâu

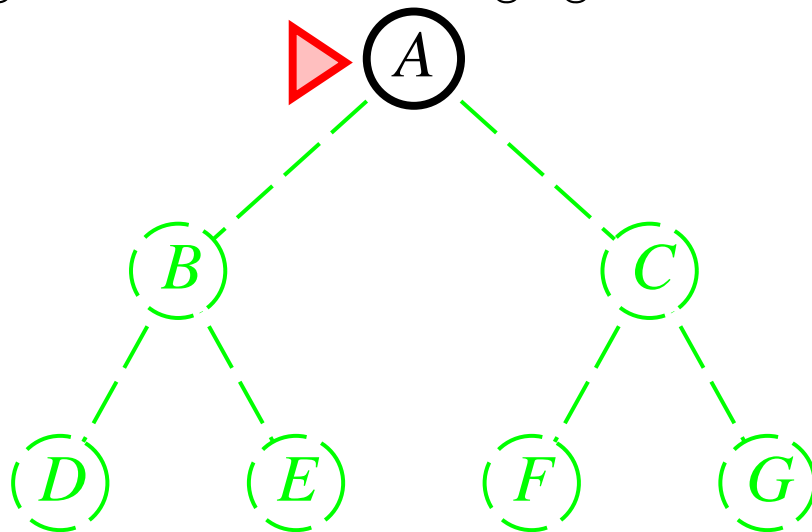
Tìm kiếm sâu hơn lặp đi lặp lại

Tìm kiếm theo chiều rộng

Mở rộng nút nông nhất chưa được mở rộng

Triển khai:

fringe là một hàng đợi FIFO, tức là những người kế nhiệm mới sẽ ở cuối

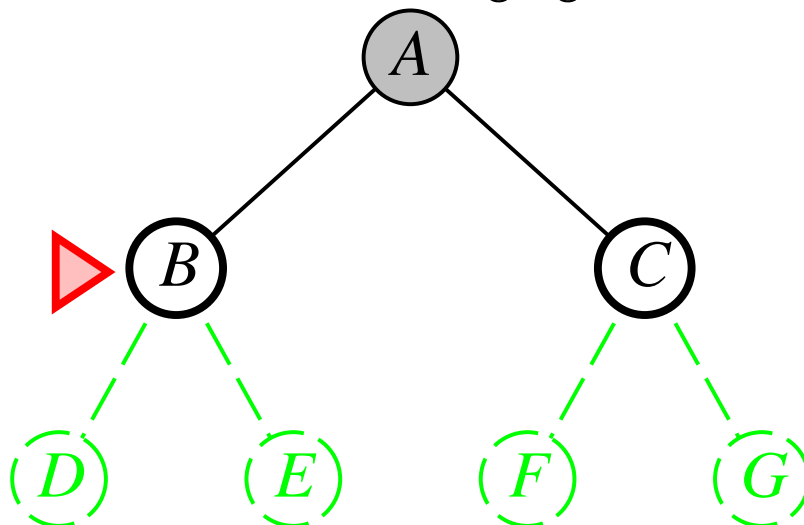


Tìm kiếm theo chiều rộng

Mở rộng nút nông nhất chưa được mở rộng

Triển khai:

fringe là một hàng đợi FIFO, tức là những người kế nhiệm mới sẽ ở cuối

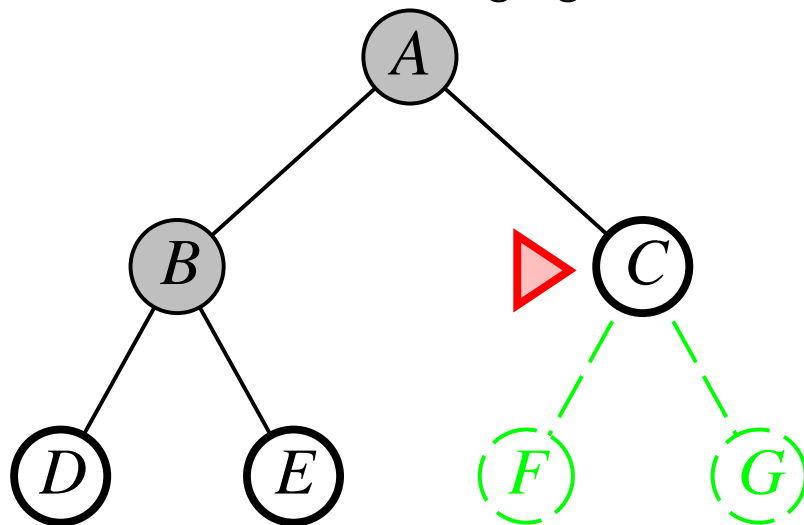


Tìm kiếm theo chiều rộng

Mở rộng nút nông nhất chưa được mở rộng

Triển khai:

fringe là một hàng đợi FIFO, tức là những người kế nhiệm mới sẽ ở cuối

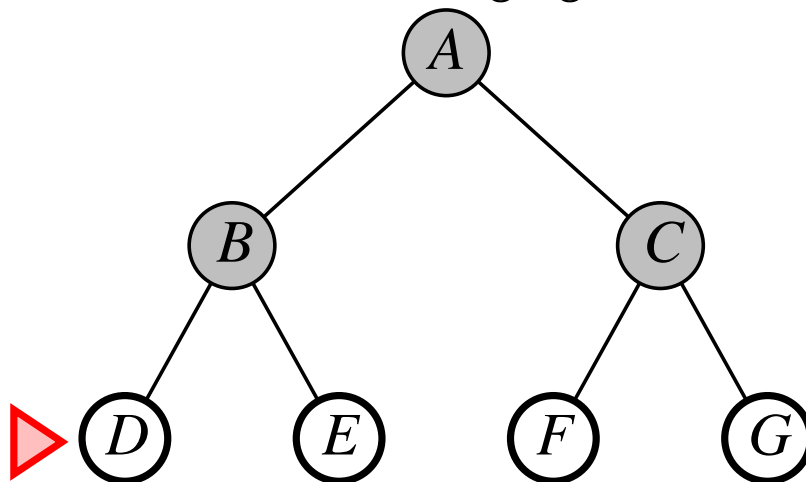


Tìm kiếm theo chiều rộng

Mở rộng nút nông nhất chưa được mở rộng

Triển khai:

fringe là một hàng đợi FIFO, tức là những người kế nhiệm mới sẽ ở cuối



Thuộc tính tìm kiếm theo chiều rộng

Hoàn thành??

Thuộc tính tìm kiếm theo chiều rộng

Hoàn thành?? Có (nếu b là hữu hạn)

Thời gian??

Thuộc tính tìm kiếm theo chiều rộng

Hoàn thành?? Có (nếu b là hữu hạn)

Thời gian?? $1 + b + b^2 + b^3 + \dots + b^d + b(b^d - 1) = O(b^{d+1})$, tức là exp. in d

Không gian??

Thuộc tính tìm kiếm theo chiều rộng

Hoàn thành?? Có (nếu b là hữu hạn)

Thời gian?? $1 + b + b^2 + b^3 + \dots + b^d + b(b^d - 1) = O(b^{d+1})$, tức là exp. in d

Space?? $O(b^{d+1})$ (giữ mọi nút trong bộ nhớ)

Tối ưu??

Thuộc tính tìm kiếm theo chiều rộng

Hoàn thành?? Có (nếu b là hữu hạn)

Thời gian?? $1 + b + b^2 + b^3 + \dots + b^d + b(b^d - 1) = O(b^{d+1})$, tức là exp. in d

Space?? $O(b^{d+1})$ (giữ mọi nút trong bộ nhớ)

Tối ưu?? Có (nếu chi phí = 1 mỗi bước); nói chung là không tối ưu

Không gian là vấn đề lớn; có thể dễ dàng tạo các nút ở tốc độ 100MB/giây

vậy 24 giờ = 8640GB.

Tìm kiếm chi phí tổng nhất

Mở rộng nút chưa được mở rộng với chi phí thấp nhất

Triển khai:

fringe = hàng đợi được sắp xếp theo chi phí đường dẫn, thấp nhất trước

Tương đương với chiều rộng đầu tiên nếu chi phí bước đều bằng nhau

Hoàn thành?? Có, nếu chi phí bước $\geq \epsilon$

Thời gian?? # nút có $g \leq$ chi phí cho giải pháp tối ưu, $O(b^{\lceil C^*/\epsilon \rceil})$

trong đó C^* là chi phí của giải pháp tối ưu

Không gian?? # nút có $g \leq$ chi phí của giải pháp tối ưu, $O(b^{\lceil C^*/\epsilon \rceil})$

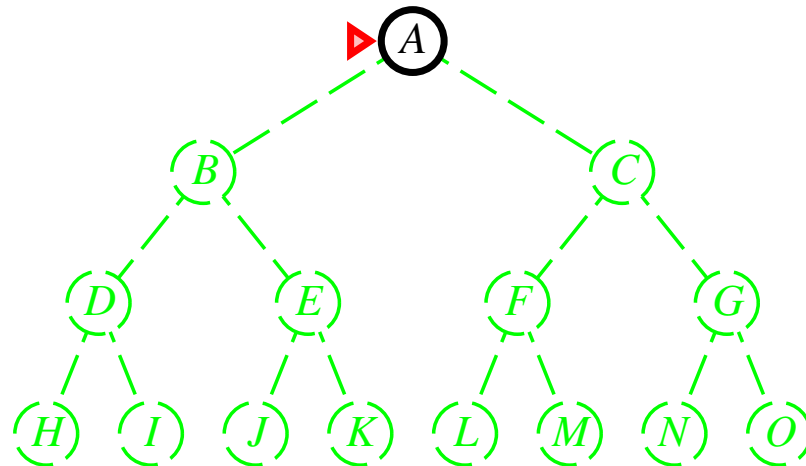
Tối ưu?? Có—các nút được mở rộng theo thứ tự tăng dần $g(n)$

Tìm kiếm theo chiều sâu

Mở rộng nút chưa được mở rộng sâu nhất

Triển khai:

fringe = Hàng đợi LIFO, tức là đặt những người kế nhiệm lên hàng đầu

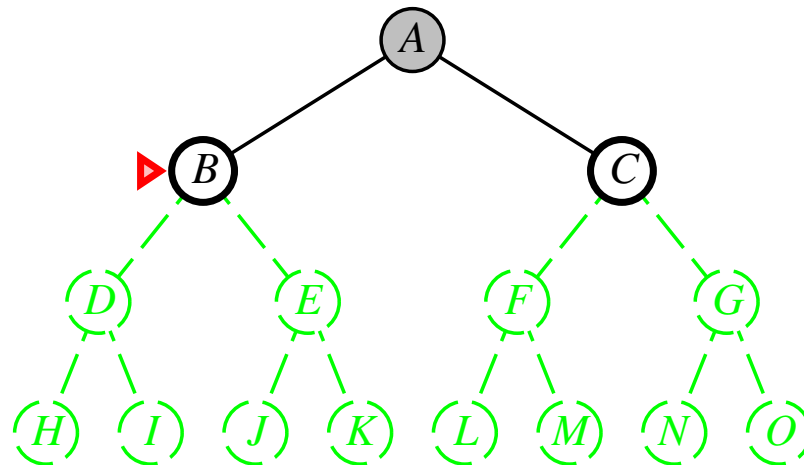


Tìm kiếm theo chiều sâu

Mở rộng nút chưa được mở rộng sâu nhất

Triển khai:

fringe = Hàng đợi LIFO, tức là đặt những người kế nhiệm lên hàng đầu

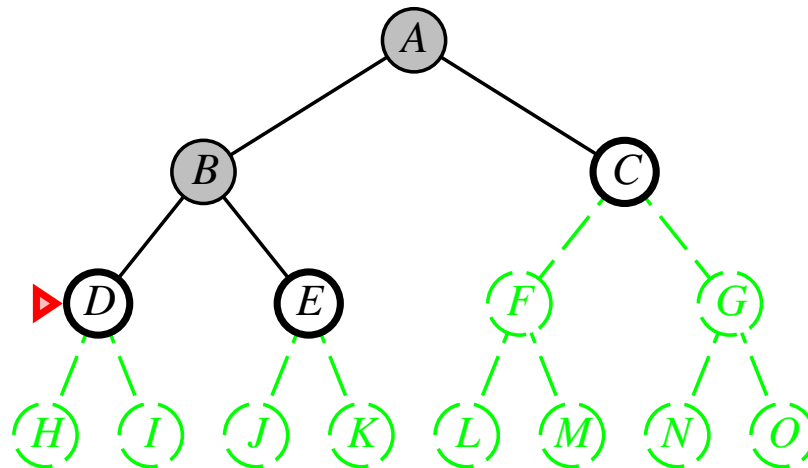


Tìm kiếm theo chiều sâu

Mở rộng nút chưa được mở rộng sâu nhất

Triển khai:

fringe = Hàng đợi LIFO, tức là đặt những người kế nhiệm lên hàng đầu

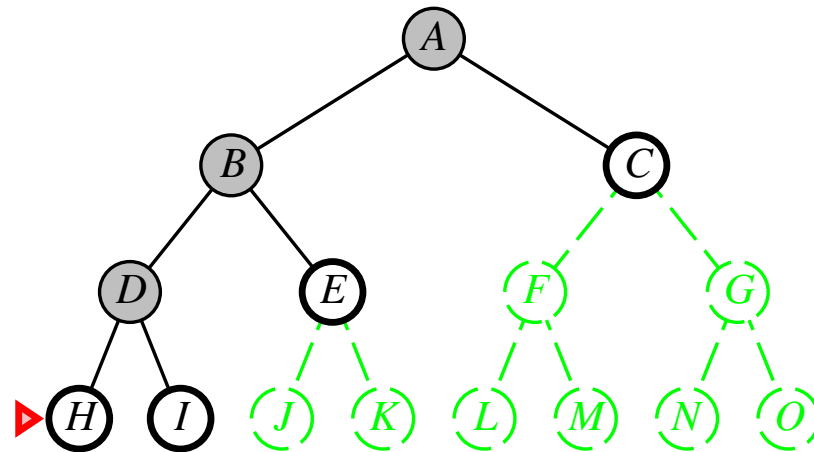


Tìm kiếm theo chiều sâu

Mở rộng nút chưa được mở rộng sâu nhất

Triển khai:

fringe = Hàng đợi LIFO, tức là đặt những người kế nhiệm lên hàng đầu

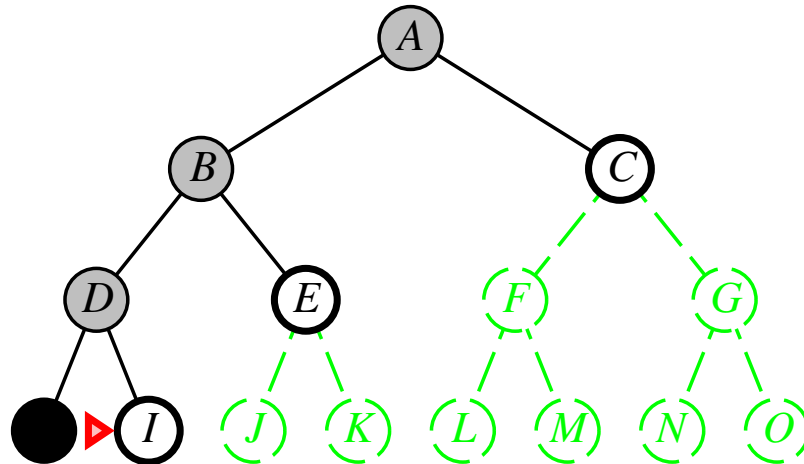


Tìm kiếm theo chiều sâu

Mở rộng nút chưa được mở rộng sâu nhất

Triển khai:

fringe = Hàng đợi LIFO, tức là đặt những người kế nhiệm lên hàng đầu

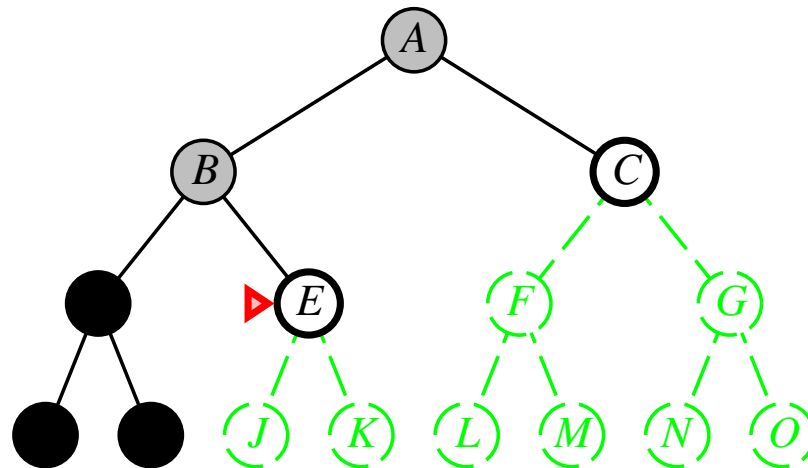


Tìm kiếm theo chiều sâu

Mở rộng nút chưa được mở rộng sâu nhất

Triển khai:

fringe = Hàng đợi LIFO, tức là đặt những người kế nhiệm lên hàng đầu

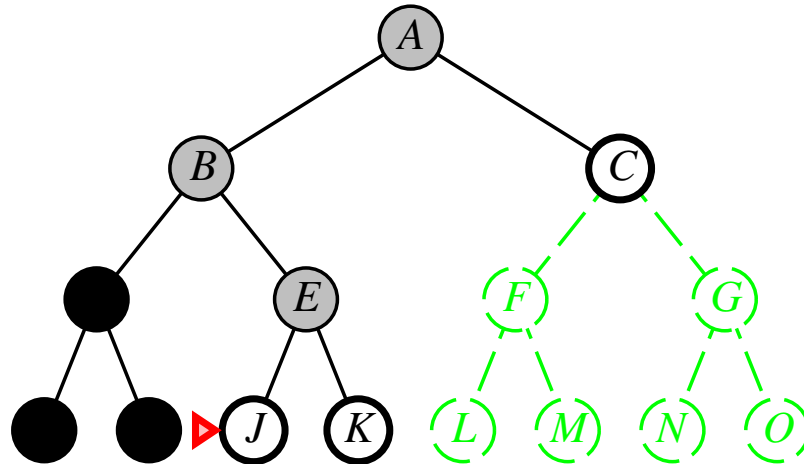


Tìm kiếm theo chiều sâu

Mở rộng nút chưa được mở rộng sâu nhất

Triển khai:

fringe = Hàng đợi LIFO, tức là đặt những người kế nhiệm lên hàng đầu

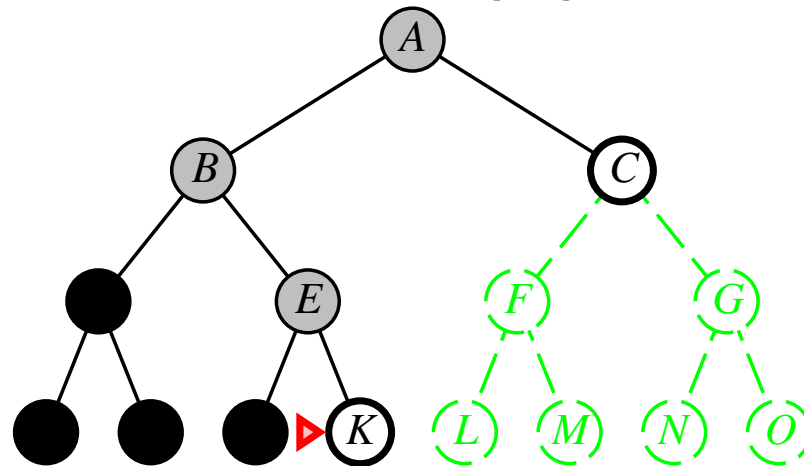


Tìm kiếm theo chiều sâu

Mở rộng nút chưa được mở rộng sâu nhất

Triển khai:

fringe = Hàng đợi LIFO, tức là đặt những người kế nhiệm lên hàng đầu

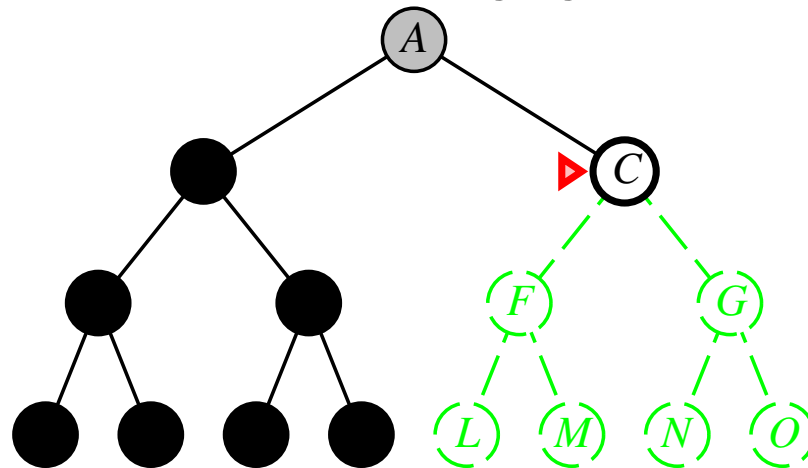


Tìm kiếm theo chiều sâu

Mở rộng nút chưa được mở rộng sâu nhất

Triển khai:

fringe = Hàng đợi LIFO, tức là đặt những người kế nhiệm lên hàng đầu

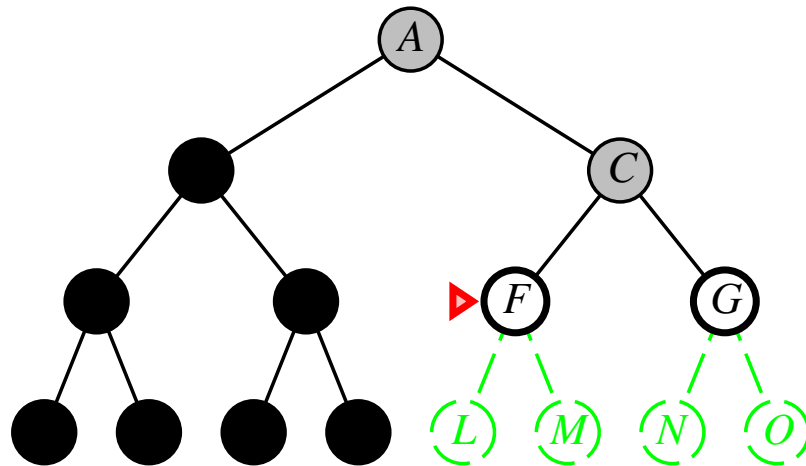


Tìm kiếm theo chiều sâu

Mở rộng nút chưa được mở rộng sâu nhất

Triển khai:

fringe = Hàng đợi LIFO, tức là đặt những người kế nhiệm lên hàng đầu

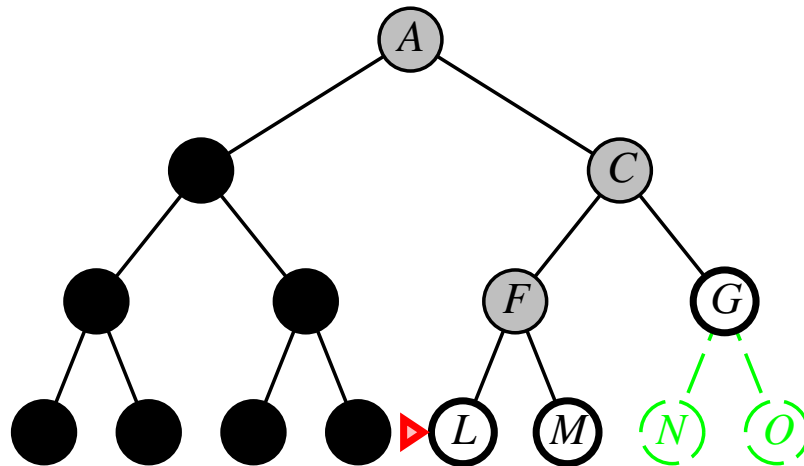


Tìm kiếm theo chiều sâu

Mở rộng nút chưa được mở rộng sâu nhất

Triển khai:

fringe = Hàng đợi LIFO, tức là đặt những người kế nhiệm lên hàng đầu

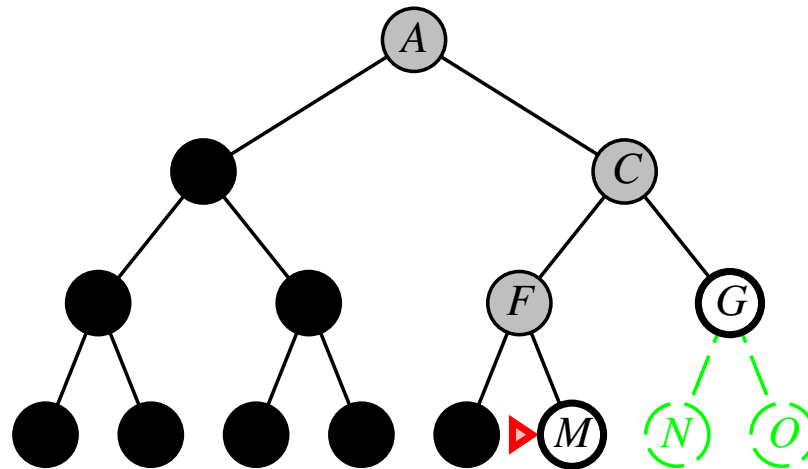


Tìm kiếm theo chiều sâu

Mở rộng nút chưa được mở rộng sâu nhất

Triển khai:

fringe = Hàng đợi LIFO, tức là đặt những người kế nhiệm lên hàng đầu



Thuộc tính tìm kiếm theo chiều sâu

Hoàn thành??

Thuộc tính tìm kiếm theo chiều sâu

Hoàn thành?? Không: thất bại trong không gian có độ sâu vô hạn, không gian có vòng lặp

Sửa đổi để tránh các trạng thái lặp lại dọc theo đường dẫn

⇒ hoàn thành trong không gian hữu hạn

Thời gian??

Thuộc tính tìm kiếm theo chiều sâu

Hoàn thành?? Không: thất bại trong không gian có độ sâu vô hạn, không gian có vòng lặp

Sửa đổi để tránh các trạng thái lặp lại dọc theo đường dẫn

⇒ hoàn thành trong không gian hữu hạn

Thời gian?? $O(b^m)$: khủng khiếp nếu m lớn hơn nhiều so với d

nhưng nếu các giải pháp dày đặc, có thể nhanh hơn nhiều so với chiều rộng đầu tiên

Không gian??

Thuộc tính tìm kiếm theo chiều sâu

Hoàn thành?? Không: thất bại trong không gian có độ sâu vô hạn, không gian có vòng lặp

Sửa đổi để tránh các trạng thái lặp lại dọc theo đường dẫn

⇒ hoàn thành trong không gian hữu hạn

Thời gian?? $O(b^m)$: khủng khiếp nếu m lớn hơn nhiều so với d

nhưng nếu các giải pháp dày đặc, có thể nhanh hơn nhiều so với chiều rộng đầu tiên

Không gian?? $O(bm)$, tức là không gian tuyến tính!

Tối ưu??

Thuộc tính tìm kiếm theo chiều sâu

Hoàn thành?? Không: thất bại trong không gian có độ sâu vô hạn, không gian có vòng lặp

Sửa đổi để tránh các trạng thái lặp lại dọc theo đường dẫn

⇒ hoàn thành trong không gian hữu hạn

Thời gian?? $O(b^m)$: khủng khiếp nếu m lớn hơn nhiều so với d

nhưng nếu các giải pháp dày đặc, có thể nhanh hơn nhiều so với chiều rộng đầu tiên

Không gian?? $O(bm)$, tức là không gian tuyến tính!

Tối ưu?? Không

Tìm kiếm giới hạn độ sâu

= tìm kiếm theo chiều sâu với giới hạn độ sâu l ,
tức là các nút ở độ sâu l không có nút kế thừa

Triển khai đệ quy:

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns soln/fail/cutoff  
  RECURSIVE-DLS(MAKE-NODE(INITIAL-STATE[problem]), problem, limit)  
  
function RECURSIVE-DLS(node, problem, limit) returns soln/fail/cutoff  
  cutoff-occurred?  $\leftarrow$  false  
  if GOAL-TEST(problem, STATE[node]) then return node  
  else if DEPTH[node] = limit then return cutoff  
  else for each successor in EXPAND(node, problem) do  
    result  $\leftarrow$  RECURSIVE-DLS(successor, problem, limit)  
    if result = cutoff then cutoff-occurred?  $\leftarrow$  true  
    else if result  $\neq$  failure then return result  
  if cutoff-occurred? then return cutoff else return failure
```

Tìm kiếm chuyên sâu lặp lại

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution
inputs: problem, a problem
for depth  $\leftarrow$  0 to  $\infty$  do
  result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)
  if result  $\neq$  cutoff then return result
end
```

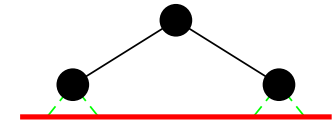
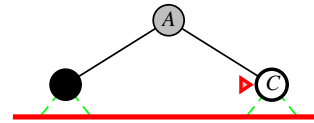
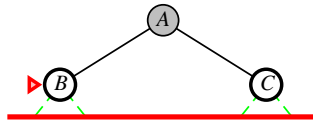
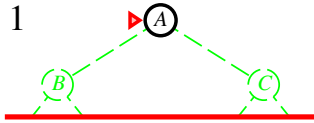
Tìm kiếm sâu lặp đi lặp lại $l = 0$

Limit = 0



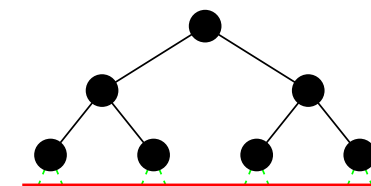
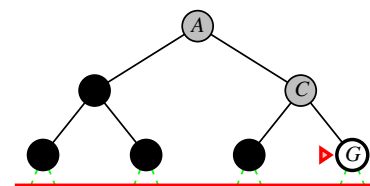
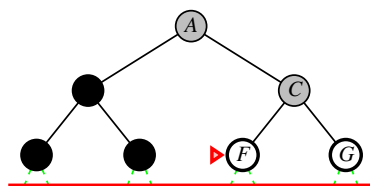
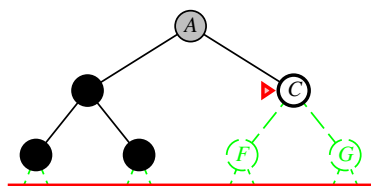
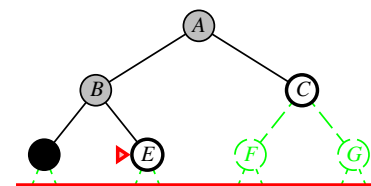
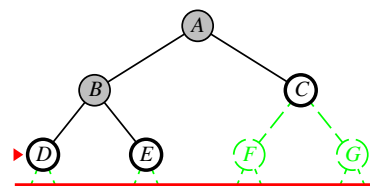
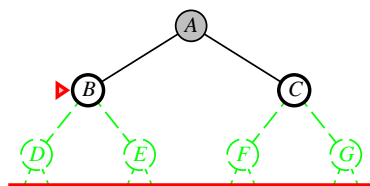
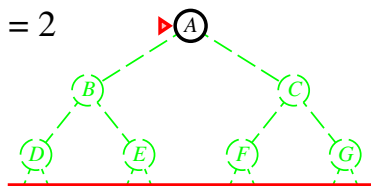
Tìm kiếm sâu lặp đi lặp lại $l = 1$

Limit = 1



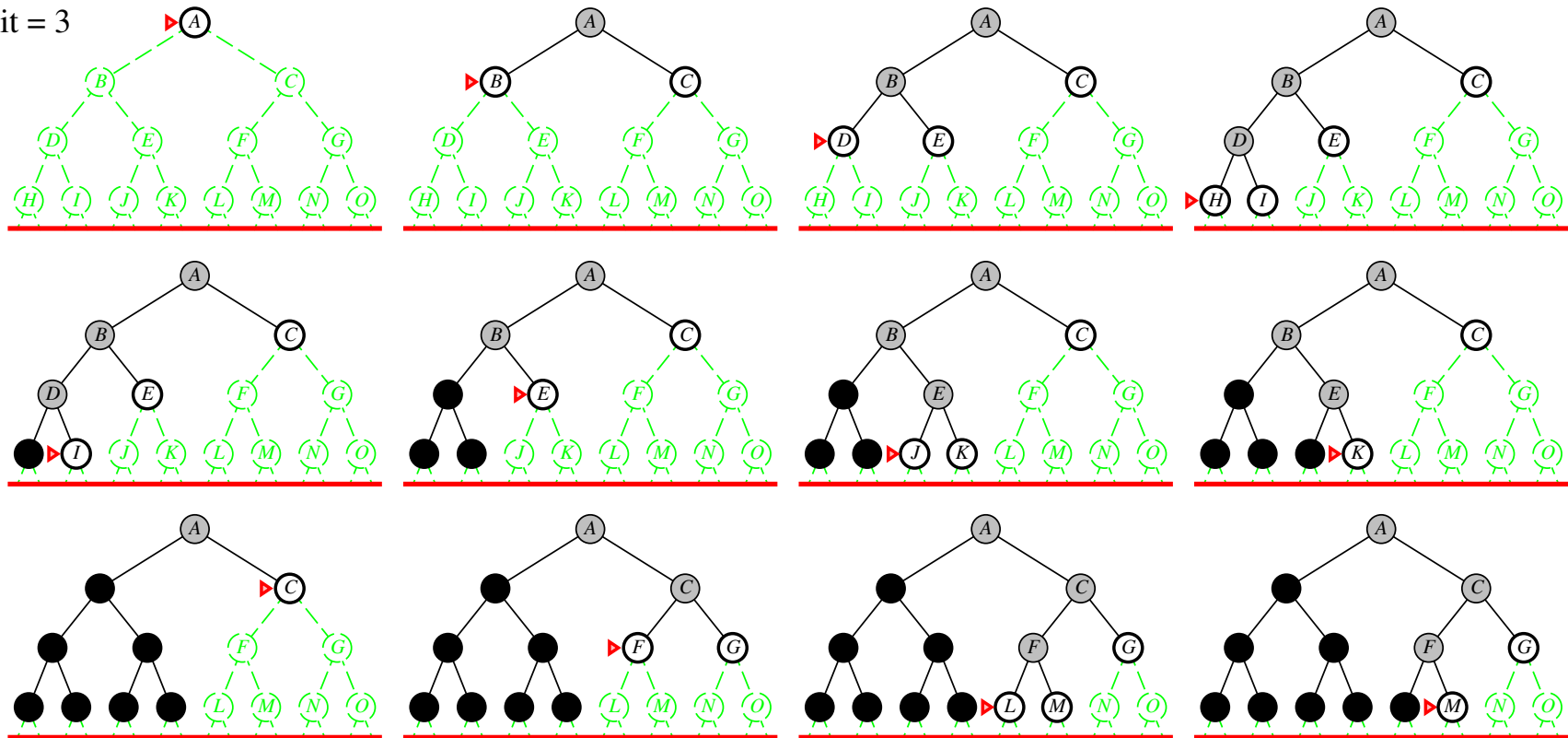
Tìm kiếm sâu lặp đi lặp lại $l = 2$

Limit = 2



Tìm kiếm sâu lặp đi lặp lại $l = 3$

Limit = 3



Thuộc tính của tìm kiếm sâu lặp đi lặp lại

Hoàn thành??

Thuộc tính của tìm kiếm sâu lặp đi lặp lại

Hoàn thành?? Có

Thời gian??

Thuộc tính của tìm kiếm sâu lặp đi lặp lại

Hoàn thành?? Có

Thời gian?? $(d + 1)b^0 + db^1 + (d - 1)b^2 + \dots + b^d = O(b^d)$

Không gian??

Thuộc tính của tìm kiếm sâu lặp đi lặp lại

Hoàn thành?? Có

Thời gian?? $(d + 1)b^0 + db^1 + (d - 1)b^2 + \dots + b^d = O(b^d)$

Không gian?? $O(bd)$

Tối ưu??

Thuộc tính của tìm kiếm sâu lặp đi lặp lại

Hoàn thành?? Có

Thời gian?? $(d + 1)b^0 + db^1 + (d - 1)b^2 + \dots + b^d = O(b^d)$

Không gian?? $O(bd)$

Tối ưu?? Có, nếu chi phí bước = 1

Có thể được sửa đổi để khám phá cây chi phí thống nhất

So sánh số cho $b = 10$ và $d = 5$, giải pháp ở lá ngoài cùng bên phải:

$$N(\text{IDS}) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450$$

$$N(\text{BFS}) = 10 + 100 + 1,000 + 10,000 + 100,000 + 999,990 = 1,111,100$$

IDS hoạt động tốt hơn vì các nút khác ở độ sâu d không được mở rộng

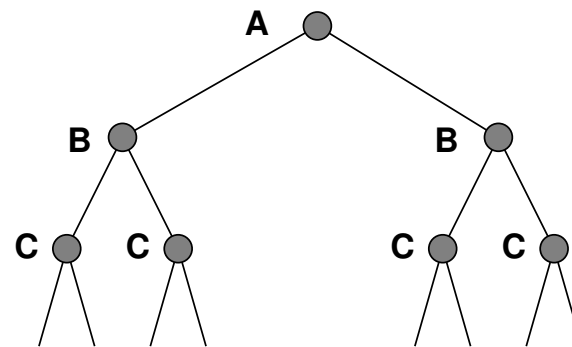
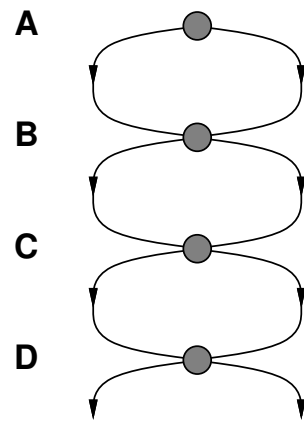
BFS có thể được sửa đổi để áp dụng kiểm tra mục tiêu khi một nút được tạo

Tóm tắt thuật toán

Criterion	Breadth- First	Uniform- Cost	Depth- First	Depth- Limited	Iterative Deepening
Complete?	Yes*	Yes*	No	Yes, if $l \geq d$	Yes
Time	b^{d+1}	$b^{\lceil C^*/\epsilon \rceil}$	b^m	b^l	b^d
Space	b^{d+1}	$b^{\lceil C^*/\epsilon \rceil}$	bm	bl	bd
Optimal?	Yes*	Yes	No	No	Yes*

Trạng thái lặp lại

Việc không phát hiện được các trạng thái lặp lại có thể biến một bài toán tuyến tính thành một bài toán số mũ!



Tìm kiếm đồ thị

function GRAPH-SEARCH(*problem*, *fringe*) **returns** a solution, or failure

closed $\leftarrow$ an empty set

fringe $\leftarrow$ INSERT(MAKE-NODE(INITIAL-STATE[*problem*]), *fringe*)

loop do

if *fringe* is empty **then return** failure

node $\leftarrow$ REMOVE-FRONT(*fringe*)

if GOAL-TEST(*problem*, STATE[*node*]) **then return** *node*

if STATE[*node*] is not in *closed* **then**

add STATE[*node*] to *closed*

fringe $\leftarrow$ INSERTALL(EXPAND(*node*, *problem*), *fringe*)

end

Tóm tắt

Việc xây dựng vấn đề thường yêu cầu trừu tượng hóa các chi tiết trong thế giới thực để xác định một không gian trạng thái có thể được khám phá

Sự đa dạng của các chiến lược tìm kiếm không chính xác

Tìm kiếm sâu hơn lặp lại chỉ sử dụng không gian tuyến tính và không mất nhiều thời gian hơn các thuật toán chưa hiểu rõ khác

Tìm kiếm đồ thị có thể hiệu quả hơn theo cấp số nhân so với tìm kiếm cây